

HoarePrompt: Structural Reasoning About Program Correctness in Natural Language

Dimitrios Stamatios Bouras

Peking University
Beijing, China

2501112125@stu.pku.edu.cn

Yihan Dai

Peking University
Beijing, China

2501112020@stu.pku.edu.cn

Tairan Wang

University College London
London, United Kingdom

tairan.wang.22@ucl.ac.uk

Yingfei Xiong

Peking University
Beijing, China

xyngyf@pku.edu.cn

Sergey Mechtaev*

Peking University
Beijing, China

mechtaev@pku.edu.cn

Abstract

While software requirements are often expressed in natural language, verifying the correctness of a program against such requirements is a hard and underexplored problem. Large language models (LLMs) are promising candidates for addressing this challenge, however, our experience shows that they are ineffective in this task, often failing to detect even straightforward bugs. To address this gap, we introduce HoarePrompt, a novel approach that adapts fundamental ideas from program verification to natural language artifacts. Inspired by the strongest postcondition calculus, HoarePrompt employs a systematic, step-by-step process in which an LLM generates natural language descriptions of reachable program states at various code points. To manage loops, we propose few-shot-driven k-induction, an adaptation of the k-induction method widely used in model checking. Once program states are described, HoarePrompt leverages the LLM to assess whether the program, annotated with these state descriptions, conforms to the natural language requirements. For evaluating the quality of classifiers of program correctness with respect to natural language requirements, we constructed CoCoLaNeL, a challenging dataset of solutions to programming competition problems. Our experiments show that HoarePrompt improves the MCC by 61% compared to directly using Zero-shot-CoT prompts for correctness classification. Furthermore, HoarePrompt outperforms a classifier that assesses correctness via LLM-based test generation by an MCC increase of 106%. The inductive reasoning mechanism contributes a 26% boost to MCC, underscoring its effectiveness in managing loops.

ACM Reference Format:

Dimitrios Stamatios Bouras, Yihan Dai, Tairan Wang, Yingfei Xiong, and Sergey Mechtaev. 2026. HoarePrompt: Structural Reasoning About Program Correctness in Natural Language. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773206>

*Sergey Mechtaev is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773206>

1 Introduction

Assessing whether a program meets its requirements is a key part of software quality assurance. In practice, these requirements are often written in natural language, geared towards human-to-human communication or designed as prompts for LLMs. Verifying program correctness against such requirements is an inherently difficult and unexplored challenge. One approach is address it is to generate tests based on requirements using an LLM, and use these tests to check program correctness [5]. More generally, natural language requirements can be translated in formal ones, and then used in conjunction with verification tools [16]. Despite the promise of these approaches, our experiments show that LLMs tend to generate incorrect or incomplete assertions when handling complex requirements, which reduces their accuracy. Another option is to directly employ LLMs to assess a program's conformance to requirements, relying on LLMs' ability to reason about both natural language and code [34]. Our experiments show that the state-of-the-art LLMs struggle with this task, often failing to identify even simple bugs. We attribute this limitation to the models' inability to reason about the intricacies of program semantics.

To address this gap, we introduce HoarePrompt, a program analysis approach for reasoning about program correctness w.r.t. natural language requirements that synergizes LLM reasoning techniques with ideas from program verification. In its core is the *natural Hoare triple*, a variant of the traditional Hoare triple [22] with pre- and postconditions expressed in natural language; for example,

$\{x \text{ is a full name}\} x = x.\text{split}()[0] \{x \text{ is a first name}\}.$

Inspired by the strongest postcondition calculus [14], HoarePrompt employs a step-by-step process to generate natural language descriptions of reachable program states at various points in the code. This is achieved by sampling *natural strongest postconditions* (NSP) from $\mathcal{SP}$, our LLM-based natural Hoare triple completion model defining the conditional probability distribution $\mathcal{SP}(\text{Post} \mid \text{Pre}, C)$, where *Pre*, *Post* are natural pre- and postconditions, *C* is a program fragment. Inspired by methods to enhance LLM reasoning about program execution [38], HoarePrompt uses these state descriptions to annotate a program, and then prompts an LLM to assess if the annotated program aligns with the given requirements.

HoarePrompt's key novelty in comparison with traditional program analysis and verification is in using natural language as the medium of reasoning about program semantics, as opposed to,

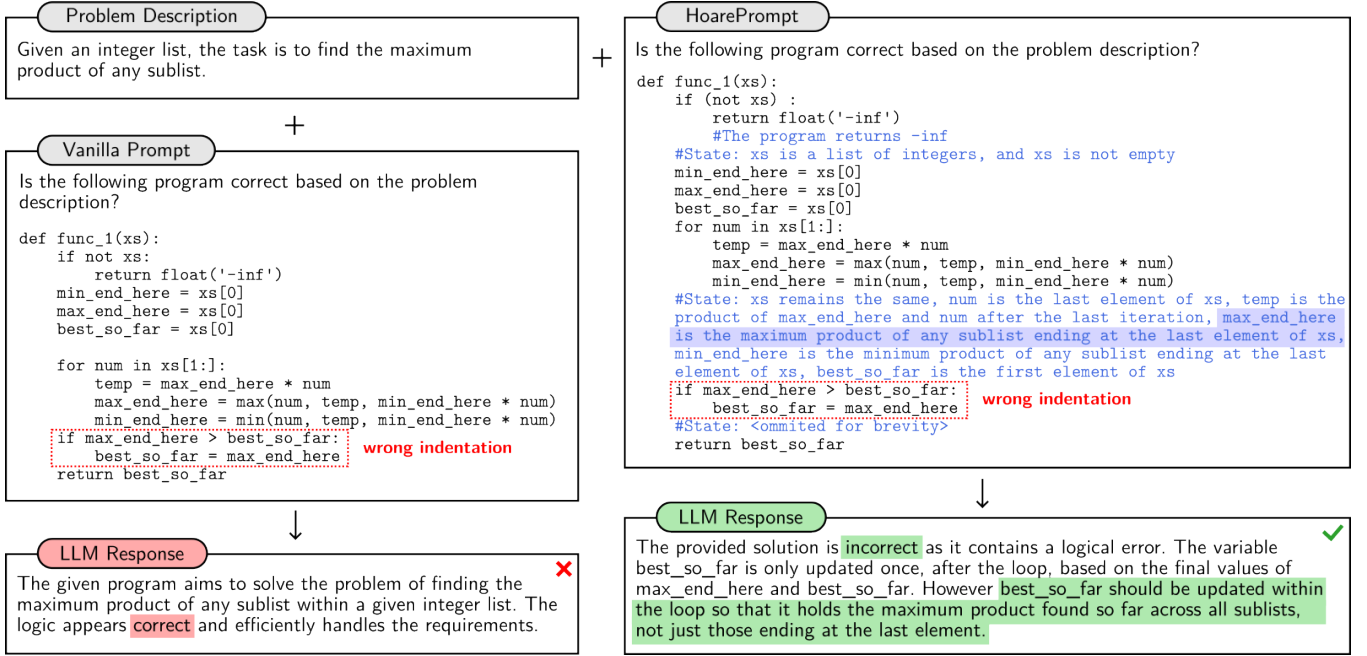


Figure 1: Annotating code with natural language descriptions of reachable program states, which are automatically inferred by HoarePrompt, enables an LLM to find a bug that is missed by the straightforward Vanilla prompt.

e.g., first-order logic [17] or monotonic functions over lattices [10]. This enables a direct application of HoarePrompt to assessing program correctness w.r.t. natural language requirements without hard and time-consuming formal modeling. In comparison to previous techniques for reasoning about code with LLMs [28, 34, 38], HoarePrompt reasons about all reachable states simultaneously instead of individual concrete states. This enables HoarePrompt to operate statically, without executing the program and without reliance on provided failing tests. In comparison with directly prompting LLMs to analyze programs, HoarePrompt simplifies this task by decomposing it into small, independent subtasks, which are solved step-by-step in the spirit of chain-of-thought reasoning [51].

Loops pose a fundamental challenge in program analysis and verification because they give rise to infinite execution paths [46]. Our experiments demonstrate that LLMs also struggle to summarize the semantics of complex loops, producing incorrect natural postconditions. To address this problem, we introduce a novel method, *few-shot-driven k-induction*, synergizing few-shot learning [4] with the idea of *k-induction* [15] widely used in model checking. This method starts with iteratively unwinding a loop k times to compute natural postconditions after each iteration. After that, it performs an induction inference step by prompting an LLM to compute the postcondition for the entire loop, using the computed natural Hoare triples for individual iterations as few-shot examples.

To evaluate HoarePrompt, we created CoCoLaNeL, a dataset of program and natural language requirement pairs, where some programs meet the requirements and others do not due to human-introduced bugs. All programs and requirements were published since January 2024, after the training cut-off date of many SOTA

Table 1: Success rates for detecting the bug in Figure 1 via prompts with and without state annotations, using four LLMs, across 60 responses for each, with the temperature 0.5.

Model	Not Annotated	Annotated
Qwen2.5-7B	0%	11.7%
Qwen2.5-Coder-32B	15%	85%
Qwen2.5-72B	35%	85%
Llama3.1-70B	50%	100%

LLMs to prevent data leakage. We formulated program correctness assessment as a binary classification problem. Our experiments show that HoarePrompt, at the expense of a higher token use, improves the Matthews Correlation Coefficient (MCC) by 61% compared to directly using Zero-shot-CoT prompts. Furthermore, HoarePrompt outperforms a classifier that assesses correctness via LLM-generated tests by increasing the MCC by 106%. The few-shot-driven *k-induction* mechanism contributes a 26% boost to MCC. We also show that HoarePrompt improves code generation when used as a feedback mechanism, outperforming both direct generation (by 19.4%) and test-based feedback (by 12.7%).

In summary, the paper makes the following contributions:

- HoarePrompt, the first program analysis approach using natural language as the medium for reasoning about program semantics.
- Few-shot-driven *k-induction*, the first approach that enhances LLM’s ability to summarize loops.
- CoCoLaNeL, a challenging benchmark for program correctness classification.

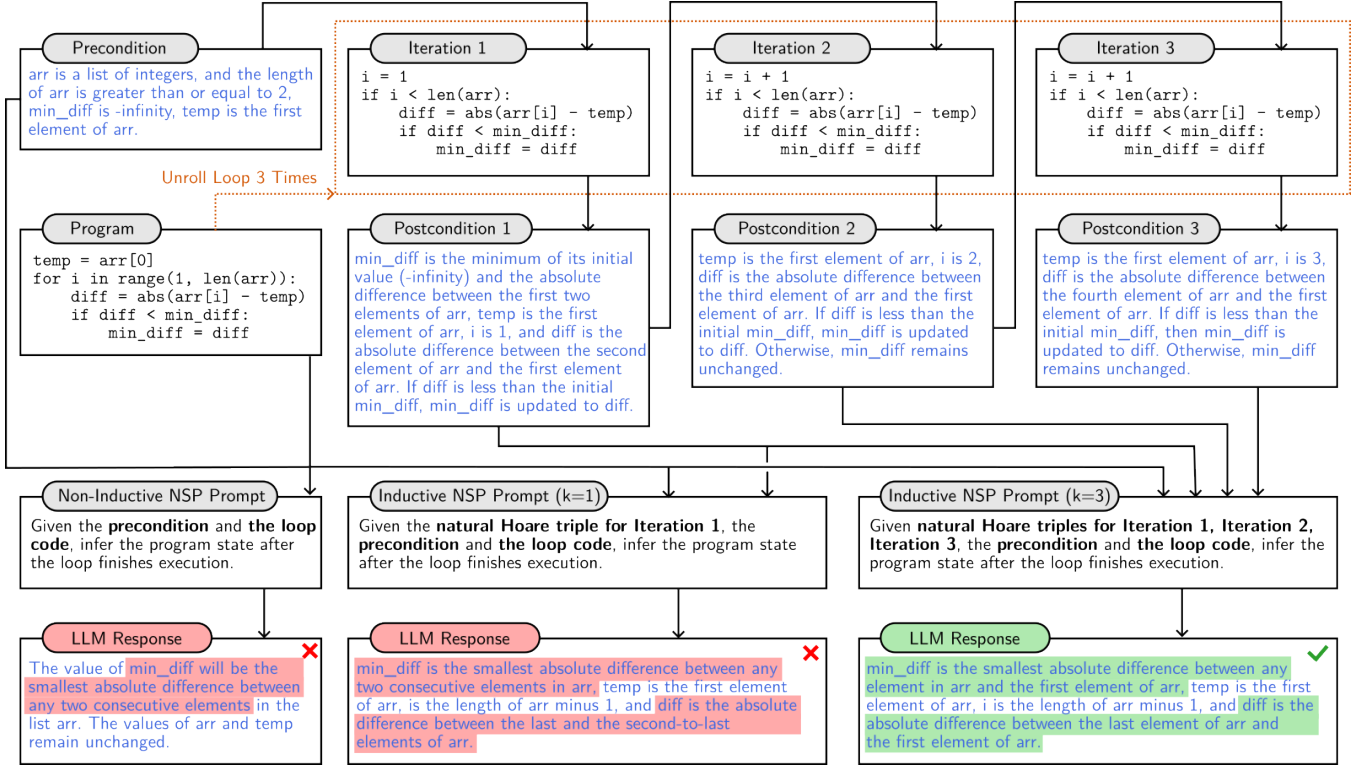


Figure 2: Few-shot-driven k -induction unrolls the loop three times, computes the natural strongest postconditions (NSP) for each iteration sequentially, and uses the resulting natural Hoare triples as few-shot examples for inductively inferring the postcondition for the entire loop, which enhances precision compared to non-inductive inference, or only a single unrolling.

- An empirical study of classifying program correctness w.r.t. natural language requirements and guiding code generation, demonstrating the effectiveness of HoarePrompt.

All code, scripts, and data, to reproduce this work are available at Figshare: <https://figshare.com/s/5d39b388bd9ff2c7ffab>.

2 Motivating Examples

To motivate our approach, we first show that annotating a program with natural language descriptions of reachable program states enhances LLM’s reasoning about program correctness, and then show how our approach precisely summarizes loops.

2.1 State Descriptions Enhance LLM Reasoning

Consider the following problem: “given an integer list, the task is to find the maximum product of any sublist.” Figure 1 (left) presents a flawed implementation of Kadane’s algorithm to solve this problem. The issue lies in incorrect indentation, which results in `best_so_far` being updated only once, after the loop ends, instead of being updated during each iteration.

Despite the bug’s simplicity, SOTA LLMs struggle to identify it consistently, and thus often wrongly classify the program as correct. Providing evidence that an LLM fails to solve a task is inherently challenging due to the model’s non-deterministic behavior [41] and its sensitivity to prompt design [44]. To address these challenges, we made every effort to present objective evidence by crafting six

distinct prompts: a straightforward (Vanilla) prompt, a Zero-shot-CoT prompt [30], an open-ended prompt [1], and combinations of these strategies, each containing the problem description and the code snippet. For each prompt, we generated 10 responses from four different LLMs, employing various temperature settings. More details about the experimental setup and results for this example can be found in the supplementary materials (Appendix).

The results in Table 1 demonstrate that LLMs, using the above six prompts, labelled collectively as “Not Annotated”, detect the bug with only a small probability. An example of an incorrect response for the Vanilla prompt is presented in Figure 1 on the left.

HoarePrompt addresses this problem as follows. It first extracts the program’s natural precondition by prompting an LLM to summarize the set of valid input values based on the problem description. In this case, the inferred precondition is `{xs is a list of integers}`. Then, it iteratively queries its natural strongest postcondition (NSP) model to propagate reachable state descriptions along the program’s control flow graph (CFG). For example, the Hoare Triplet for the first if statement:

```
{xs is a list of integers}
if (not xs):
    return float('-inf')
{xs is a list of integers, and xs is not empty}
```

capturing the fact that if an execution reaches the program point after the if statement, the list is guaranteed to be non-empty.

HoarePrompt continues to propagate the states until it reaches the end of the program. Loops, which present a particular challenge, are handled via inductive reasoning as illustrated in Section 2.2.

Finally, HoarePrompt annotates the program with the reachable states at key program points (return statements, after top-level if statements and loops) as presented in Figure 1 on the right. Adding such annotations to the six prompts significantly increases LLM’s success rate of correctness classification as shown in Table 1. This improvement stems from HoarePrompt’s compositional reasoning strategy. Since it reasons about individual blocks in isolation, the loop is interpreted based only on its local semantics. In this case, HoarePrompt accurately captures that the loop updates only consider sublists ending at the final element, with the resulting postcondition annotation enabling better error detection.

2.2 Inductively Reasoning About Loops

Reasoning about loops is challenging, because they induce infinite behaviours [46], and our experience shows that LLMs also struggle to summarize them. To alleviate this problem, we take inspiration for the k -induction technique [15] widely used in model checking, which first checks that the property holds for all states in the first k steps of the program’s execution, and then proves that if a property holds for any k consecutive steps, then it holds for the next step.

To illustrate our adaptation of k -induction, which we refer to as *few-shot-driven k -induction*, consider the program with a loop in Figure 2, labelled as “Program”. Assume that its precondition is given in “Precondition”. Inferring its natural strongest postcondition (NSP) using the straightforward “Non-Inductive NSP Prompt” with Qwen2.5-72B results in an incorrect natural postcondition mistakenly stating the `min_diff` is the smallest absolute difference between any two consecutive elements in the list.

HoarePrompt unrolls the loop k times, with the semantics of each iteration represented by code snippets in “Iteration 1”, ..., “Iteration k ”. Then, it sequentially computes natural strongest postconditions for each iteration, using the postcondition of the previous iteration as the precondition of the next one (the first iteration uses the precondition of the entire loop). The resulting triples

```
{Precondition} Iteration 1 {Postcondition 1},
...
{Postcondition k-1} Iteration k {Postcondition k}
```

are used as few-shot examples to infer the postcondition of the entire loop in “Inductive NSP Prompt ($k=3$)”. The intuition of this step is that LLMs generalize from few relevant examples [4], and the examples above are relevant by construction. As shown in the corresponding LLM response, the enhanced prompt enabled the LLM to correctly infer that `min_diff` is the smallest absolute difference between any element in the list and the first element.

Interestingly, with $k=1$, the mistake persisted, as the model failed to generalize from a single example. Based on our pilot study (Section 5.2), we chose $k=3$ as the best performing configuration.

2.3 Expressive Power of Natural Language

LLMs exhibit limitations in symbolic reasoning due to their reliance on natural language [48, 55]. Consequently, HoarePrompt avoids formal logical inference, opting instead for natural language as the

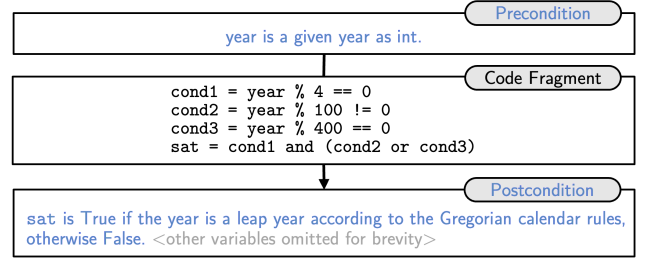


Figure 3: HoarePrompt succinctly describes program behavior in human-centric terms, which helps align formal semantics with informal requirements.

reasoning medium. While this approach does not provide formal correctness guarantees, it generates structured informal proofs, which, as demonstrated in Section 5, improve the LLMs’ ability to effectively align program semantics with informal requirements.

The method’s effectiveness stems from natural language’s expressivity. In Figure 3, Qwen2.5-72B derives an NSP that succinctly describes `sat` in human terms, beyond formal semantics—an expressivity symbolic logic lacks. Further examples are in the appendix.

3 Background

Hoare logic [22] is a formalism for reasoning about program correctness, at the core of which are specifications called *Hoare triples* $\{Pre\} C \{Post\}$, meaning that the postcondition `Post` holds in any state reachable by executing the code `C` from an initial state in which the precondition `Pre` holds, provided that the code terminates. Pre- and postconditions are predicates over program states, that are typically expressed in first-order logic [17]. Strongest postcondition calculus [14] automates verification using Hoare logic. The strongest postcondition (SP) of a program `C` w.r.t. a precondition `Pre`, denoted as $sp(Pre, C)$, is the most precise logical formula describing all possible states reachable after executing `C` from a state satisfying `Pre`. Formally,

$$\models \{Pre\} C \{Post\} \quad \text{iff} \quad sp(Pre, C) \implies Post.$$

The following algorithm for computing SP is defined compositionally for WHILE [40], a simple programming language that has only assignments ($x := e$), sequences ($S_1; S_2$), if statements, one loop instruction (while) and a single type (integer):

$$\begin{aligned}
 sp(Pre, x := e) &= Pre[x \mapsto e] \\
 sp(Pre, S_1; S_2) &= sp(sp(Pre, S_1), S_2) \\
 sp(Pre, \text{if } b \text{ then } S_1 \text{ else } S_2) &= sp(Pre \wedge b, S_1) \vee sp(Pre \wedge \neg b, S_2) \\
 sp(Pre, \text{while } b \text{ do } S) &= (Pre \wedge \neg b) \vee sp(sp(Pre \wedge b, S), \text{while } b \text{ do } S)
 \end{aligned}$$

The last rule, which handles loops, is recursively defined, making it hard to compute.

The k -induction method [15] applies mathematical-induction principles to practical loop analysis. Let $P(s_i)$ denote the target property at loop state s_i after i iterations. The method has two components: the *base case* (1), which checks that the property holds for the first k iterations, and the *inductive step* (2), which proves that any window of k satisfying iterations implies the next one also

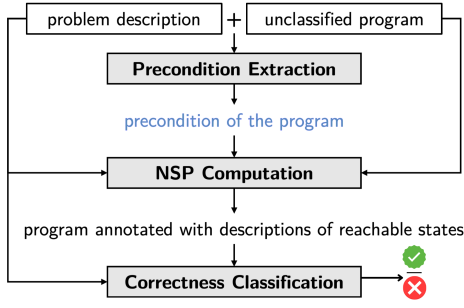


Figure 4: HoarePrompt's correctness classification workflow.

satisfies it:

$$(1) \bigwedge_{i=0}^{k-1} P(s_i) \quad (2) \forall n \geq 0. \left(\bigwedge_{i=0}^{k-1} P(s_{n+i}) \right) \rightarrow P(s_{n+k})$$

4 HoarePrompt

We introduce the *natural Hoare triple*, a Hoare triple where pre- and postconditions are expressed in natural language (NL). To differentiate natural Hoare triples from traditional Hoare triples, we highlight sentences from NL with blue:

$$\{ \text{Pre} \} C \{ \text{Post} \}.$$

In analogy with the strongest postcondition calculus, we introduce the *natural strongest postcondition (NSP)*. The traditional algorithm $\text{sp}(\text{Pre}, C)$ cannot be applied in the context of NL. Instead, we create $\mathcal{SP}$, an LLM-based natural Hoare triple completion model defining a conditional probability distribution

$$\mathcal{SP}(\text{Post} \mid \text{Pre}, \text{Stmt}).$$

Let $\llbracket \cdot \rrbracket : \text{NL} \rightarrow 2^S$ capture the human interpretation of NL state descriptions as a mapping from sentences to sets of program states S . Then, the goal to achieve by $\mathcal{SP}$ to ensure plausible reasoning can be measured using the following objective function:

Definition 4.1 (NSP Objective Function). The objective function for $\mathcal{SP}$ is the cross-entropy loss between predicted postconditions and the traditional strongest postconditions computed over the interpretations of natural language preconditions:

$$\mathcal{L} = - \sum_{\text{Post} \in \text{NL}} \mathbb{I}(\llbracket \text{Post} \rrbracket \equiv \text{sp}(\llbracket \text{Pre} \rrbracket, C)) \log \mathcal{SP}(\text{Post} \mid \text{Pre}, C),$$

where $\mathbb{I}(\cdot)$ is the indicator function.

HoarePrompt applies $\mathcal{SP}$ to classify program correctness w.r.t. natural language requirements using the workflow depicted in Figure 4. First, it prompts an LLM to extract a precondition (a description of valid program inputs) from the requirements. Second, it traverses the program's control-flow graph (CFG), while sampling natural strongest postconditions from $\mathcal{SP}$, starting from the precondition, for each code segment. Finally, it annotates key program points with the inferred reachable state descriptions, and prompts an LLM to check whether the code meets the requirements.

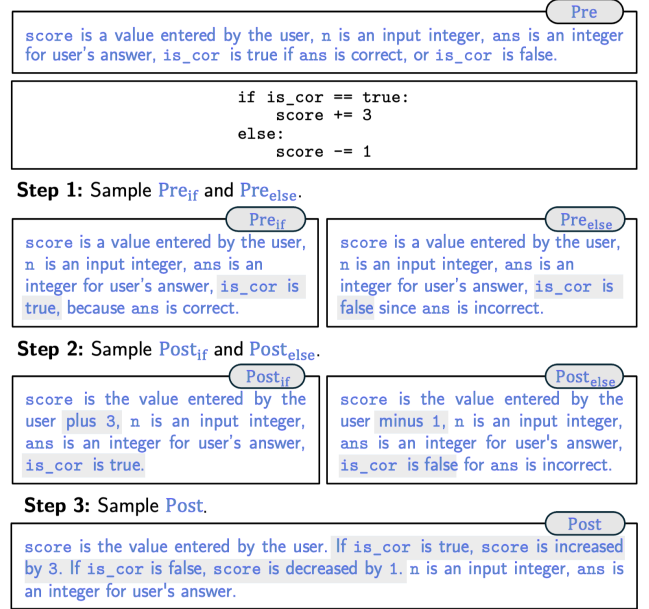


Figure 5: Computing NSP for an if-else statement.

4.1 Precondition Extraction

HoarePrompt prompts an LLM to extract a precondition from the given requirements in order to identify the set of valid program inputs. For example, for the problem description

“given N , write a function that finds the N th Fibonacci number”

and the function signature `def func(num)`, the LLM produces the precondition: `{num is a non-negative integer}`.

We implement precondition extraction using few-shot learning (see Appendix)

4.2 NSP Computation

In principle, $\mathcal{SP}$ can be realized by fine-tuning an LLM to optimize the NSP objective function (Definition 4.1). Due to budget constraints, we opted instead to adapt an LLM through prompt engineering and in-context learning. Specifically, we curated a pilot dataset, distinct from our evaluation set, and carefully designed prompts and few-shot examples for Python programs by eyeballing the LLM's output on this dataset.

Simple Statements. Atomic operations such as assignments are processed by directly prompting an LLM to compute the postcondition. We use a dedicated prompt with few-shot examples, presented in supplementary materials (Appendix), that tasks the model to infer the updated state based on variable modifications while preserving relevant information from the precondition. Apart from that, we use dedicated prompts to capture effects of return and print commands, since program correctness often depends on them.

Compound Statements. Similar to the traditional sp , $\mathcal{SP}$ is defined compositionally, which enables it to precisely reason about complex programs by breaking them down into simpler fragments. For example, given a sequence of statements $S_1; S_2$, $\mathcal{SP}$ is computed

by first completing natural Hoare triple $\{Pre\}S_1\{Post\}$, and then its postcondition $Post$ serves as the precondition for S_2 . To reduce the number of LLM queries, we group a configurable number of atomic statements together using a dedicated prompt (in Appendix).

If Statements. Figure 5 shows how if-else statements are processed by HoarePrompt. Given a precondition $\{Pre\}$ and a statement $if\ b: S_1\ else: S_2$, it first samples preconditions for the branches:

$$Pre_{if} \sim \mathcal{SP}(\cdot \mid Pre, \text{assert } b), \quad Pre_{else} \sim \mathcal{SP}(\cdot \mid Pre, \text{assert } \neg b)$$

Then, it samples NSPs for each branch:

$$Post_{if} \sim \mathcal{SP}(\cdot \mid Pre_{if}, S_1), \quad Post_{else} \sim \mathcal{SP}(\cdot \mid Pre_{else}, S_2)$$

Finally, it uses a dedicated prompt to merge $Post_{if}$ and $Post_{else}$ into a single postcondition $Post$, which will hold after the loop. Prompts used for these operations are given in the Appendix (Appendix).

Loop Statements. HoarePrompt reasons about loops using our few-shot-driven k -induction method. For a while statement $while\ b: S$ and a precondition Pre , it first samples a precondition for the loop body, and a postcondition for the first iterations:

$$Pre_{while} \sim \mathcal{SP}(\cdot \mid Pre, \text{assert } b), \quad Post_1 \sim \mathcal{SP}(\cdot \mid Pre_{while}, S).$$

Then, it continues to unwind the loop $k - 1$ times, sequentially computing the following natural pre- and postconditions for each iteration:

$$Pre_i \sim \mathcal{SP}(\cdot \mid Post_{i-1}, \text{assert } b), \quad Post_i \sim \mathcal{SP}(\cdot \mid Pre_i, S),$$

where $i \in [2, 3, \dots, k]$.

Finally, the NSP for the whole while statement is computed using our inductive prompt that includes the natural Hoare triples

$$\left\{ \{Pre_i\} S \{Post_i\} \right\}_{i \in [1, 2, \dots, k]}$$

as few-shot examples. This process is illustrated in Figure 2 for a for statement, which is handled accordingly.

Many programming languages allow iterations over a collection, e.g., the for loop in Python. The challenge of handling such iteration constructs is that they are implemented using implicit iterators, which store internal states updated at each iteration. Since iterators are not explicit variables, LLM tends to not include them when producing NSPs. Therefore, when dealing with iterators, we also explicitly model the state of the iterator.

Our modeling of Python's iterators involves maintaining the index of the next element of the iterator, which increments by 1 with each call to `next()`. Our natural language descriptions of program states refer to elements from the iterator as "the {index}th elements of the iterator", where "{index}" represents our modeled index, and the description of the iterator itself is always accompanied with the description of the current index representing its state. If an iterator is used across loops, its index also needs to be summarized via k -induction together with other parts of the state.

For a loop `for item in I`, to accurately reflect the additional changes of both the iterator and the index for at the start of each iteration, the precondition for the i -th iteration is sampled as:

$$Pre_i \sim \mathcal{SP}(\cdot \mid Post_i, \text{stmt}_{i\text{iter}}),$$

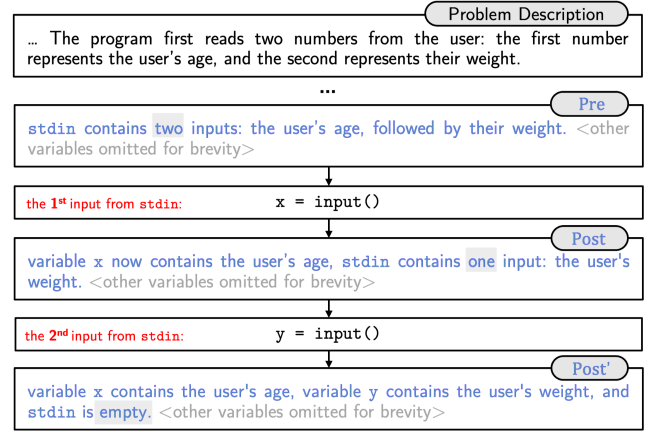


Figure 6: Computing NSP for the call of `input()` for `stdin`.

where $i \in [1, 2, \dots, k]$, $Post_0 = Pre$, $\text{stmt}_{i\text{iter}}$ is

```
try:
    item = next(I_iter)
    index += 1
except StopIteration:
    break
```

where `I_iter` is an iterator obtained from `I` and `index` is initialized to 0. For commonly used loops over lists or ranges of elements such as `range(0, n)`, we directly refer to the index of list without modeling the semantics of `next()` for simplicity.

The supplementary materials (Appendix) include our prompts for computing preconditions of the loop body, for handling while and for loops transitions, and for inductive inference.

Exceptions Handling. Given Pre , and a try-except statement

`try:  $\alpha$  except  $E_1$ :  $\beta_1$  except  $E_2$ :  $\beta_2$ , ... except  $E_m$ :  $\beta_m$ ,`

we first sample the NSP for the try-block:

$$Post_{try} \sim \mathcal{SP}(\cdot \mid Pre, \alpha),$$

and then the NSPs for different exception handling blocks:

$$Post_i \sim \mathcal{SP}(\cdot \mid \text{"variables may take arbitrary values"}, \beta_i).$$

Finally, we merge $Post_{try}, Post_1, Post_2, \dots, Post_m$ using a prompt provided in the Appendix. Note that, due to the complexity of handling nonlinear control flow, the NSP computation for exception handling blocks does not inherit any preconditions here, assuming that the state may be arbitrary.

Input Stream. Figure 6 shows how data from the standard input (`stdin`) is processed by HoarePrompt. When programs read from `stdin`, the precondition extraction explicitly describes the expected structure and content of the `stdin` based on the given problem description. So, the precondition would clearly indicate that `stdin` contains these inputs in that specific order. Every time a postcondition for a statement reading `stdin` is sampled as

$$Post \sim \mathcal{SP}(\cdot \mid Pre, x = \text{input}()),$$

the description of the stream state in $Post$ will be updated accordingly. After all data is read, the state of the stream is described

as “stdin is empty”. Note that this modeling applies only to non-interactive programs, where `stdin` is predetermined and does not depend on the execution of the program.

4.3 Correctness Classification

Let R be the natural language requirement, P be the program to be evaluated, $D = \{(R_1, P_1, y_1), (R_2, P_2, y_2), \dots, (R_n, P_n, y_n)\}$ be a labeled dataset, where $y_i \in \{\text{Correct}, \text{Incorrect}\}$ represents whether program P_i satisfies requirement R_i . The problem of classifying correctness w.r.t. natural language requirements can be formalized as the task of learning a function $f: (R, P) \rightarrow \{\text{Correct}, \text{Incorrect}\}$.

In this work, we investigate several classification functions based on NSP, as well as alternative ideas.

HoarePrompt: Given R and P , HoarePrompt first annotates key program points with descriptions of reachable states computed using $\mathcal{SP}$. Specifically, it adds annotations as comments after top-level if and loop statements. Apart from that, it adds descriptions of returned and printed values after return and print statements, as these values are often used to assess correctness. An example of an annotation program is given in Figure 1. The annotated program is passed alongside R to an LLM via our classification prompt. The Appendix contain the classification prompts for single- and multi-program programs.

HoarePrompt (No Unroll): We also implemented a variant of HoarePrompt that employs a non-compositional approach to computing the NSP for loops by prompting an LLM to infer the postcondition of the entire loop directly within a single LLM query. The prompts applied in this approach are given in the Appendix.

Vanilla: Given R and P , it directly asks an LLM to classify program correctness using the prompt given in the Appendix.

Zero-shot-CoT: Our Zero-shot-CoT [30] prompt extends the Vanilla by triggering chain-of-thought reasoning [51] by adding the “think step-by-step” instruction.

Tester: As an alternative to static reasoning, we also employed testing for classifying program correctness. Specifically, this method first generates tests based on R with an LLM using the testing prompt adapted from AgentCoder [24], executes P on the generated tests, and classifies it as correct iff it passes the tests.

5 Evaluation

Our experiments address these research questions:

- RQ1:** How does the classification performance of HoarePrompt compare to other techniques?
- RQ2:** How does few-shot-driven k -induction impact HoarePrompt’s effectiveness?
- RQ3:** How does the LLM token use of HoarePrompt compare to other techniques?
- RQ4:** How effective is HoarePrompt as a feedback mechanism in a downstream code generation task?

We selected four representative open-weight LLMs. Three come from the Qwen [19] family: Qwen2.5-7B-Instruct, a popular smaller model, Qwen2.5-Coder-32B-Instruct, a model specialized for coding tasks, and Qwen2.5-72B-Instruct, demonstrating performance comparable to GPT-4o-mini [47]. Another one is from the Llama family: Llama3.1-70b-Instruct. To measure classification quality, we adopted the Matthews Correlation Coefficient (MCC) as it considers

Table 2: Effect of unroll bound k on MCC and token usage (normalized over $k=0$).

Unroll Bound k	MCC	Token Increase
0 (no unroll)	0.211	1.00×
1	0.228	3.02×
3	0.262	11.29×
5	0.231	24.53×

all four components of the confusion matrix, and produces more reliable statistical results than F1 score [9]. Further information on MCC is available in supplementary materials (Appendix). We also report other standard metrics such as Balanced Accuracy, True Positive Rate (TPR), False Negative Rate (FNR), and False Positive Rate (FPR). To ensure robustness against LLM non-determinism, we repeated each experiment multiple times. Following best practices [41, 53], we calibrated the rerun count based on model confidence and consistency. This yielded 3 reruns per experiment. Details of this derivation are included in supplementary materials (Appendix). All experiments were conducted in a virtual machine with a AMD EPYC 7543 64-bit CPU, two cores and 16GB of RAM.

5.1 CoCoClaNeL Dataset

Although there exist numerous datasets of natural language requirements [2, 21], some including incorrect implementations [49], we could not use most of them due to potential data leakage, since memorization of correct solutions will significantly impact LLM reasoning [13, 52]. Apart from that, we required non-trivial requirements and subtle bugs to thoroughly evaluate the ability of LLMs to reason about program correctness. We constructed a benchmark CoCoClaNeL (Code Correctness Classification w.r.t. Natural Language requirements). It consists of 645 problem-solution pairs from Codeforces programming contests that took place in the first half of 2024, with solutions labelled as correct or incorrect by the Codeforces’ internal testing system. Totally, 322 of the solutions (49.9%) are incorrect. All problems and solutions are released since January 2024, after the cut-off data of many SOTA LLMs, and the problems are challenging, have unambiguous descriptions, and solutions to these problems are rigorously tested to find subtle bugs [25]. To capture subtle and challenging coding errors, we specifically included incorrect submissions that immediately preceded correct ones, hypothesizing that these would likely contain nuanced mistakes. Further details about the dataset are provided in supplementary materials (Appendix).

5.2 Hyperparameters

We tuned the unrolling bound $k \in \{0, 1, 3, 5\}$ using Llama3.1-70B on a 140-entry pilot set randomly sampled from the same source as CoCoClaNeL, but disjoint from the 645 CoCoClaNeL programs. As shown in Table 2, $k=3$ achieves the best MCC (0.262), balancing generalization and token cost. Higher k increase token usage while degrading the quality. Notably, no matter the choice of k HoarePrompt outperforms the baseline classifiers by at least 23.7%.

Another key design choice is how many and which annotations to include in the final prompt. We mark only major program points—after loops, branches, returns, prints, and try-catch

Table 3: Performance of classifiers of program correctness w.r.t. natural language requirements and their token use across four LLMs evaluated on CoCoCLaNeL. HoarePrompt significantly improves the MCC in comparison with baselines at the expense of a higher token use. HoarePrompt without loop unrolling represents a trade-off between classification quality and token cost.

Model	Classifier	Correctness Classification Quality Metrics						Tokens (Millions)		
		MCC	Δ MCC	BA	TPR	FNR	FPR	IT	OT	TT
Qwen2.5-7B	Vanilla	0.055		0.521	0.193	0.807	0.151	1.8	0.8	2.6
	Zero-shot-CoT	0.085	+54.5%	0.543	0.563	0.437	0.478	2.1	1.3	3.4
	Tester	0.109	+98.18%	0.519	0.051	0.949	0.012	1.7	3.8	5.5
	HoarePrompt	0.159	+189.1%	0.577	0.700	0.300	0.546	205.1 (93.9)	31.9	237.1 (125.9)
	HoarePrompt no unroll	0.119	+116.36%	0.558	0.666	0.334	0.550	19.9 (12.3)	3.4	23.4 (15.7)
Qwen2.5-Coder-32B	Vanilla	0.137		0.553	0.233	0.767	0.128	1.9	0.9	2.8
	Zero-shot-CoT	0.123	-10.22%	0.560	0.438	0.562	0.318	2.4	1.5	3.9
	Tester	0.096	-29.93%	0.518	0.055	0.945	0.019	1.9	3.3	5.2
	HoarePrompt	0.231	+68.61%	0.616	0.593	0.407	0.362	186.5 (88.2)	42.6	229.1 (130.8)
	HoarePrompt no unroll	0.158	+15.04%	0.579	0.571	0.429	0.412	19.2 (11.6)	4.4	23.6 (15.9)
Qwen2.5-72B	Vanilla	0.169		0.578	0.382	0.618	0.226	1.7	0.9	2.6
	Zero-shot-CoT	0.193	+14.20%	0.593	0.714	0.286	0.527	2.0	1.3	3.2
	Tester	0.131	-22.49%	0.522	0.052	0.948	0.007	1.9	3.8	5.6
	HoarePrompt	0.259	+53.25%	0.629	0.667	0.333	0.409	163.5 (78.7)	32.5	196.0 (111.2)
	HoarePrompt no unroll	0.231	+36.69%	0.615	0.618	0.382	0.387	19.3 (11.8)	3.8	23.1 (15.6)
Llama3.1-70b	Vanilla	0.161		0.579	0.671	0.329	0.513	1.7	0.01	1.7
	Zero-shot-CoT	0.157	-2.48%	0.575	0.728	0.329	0.514	1.8	0.7	2.5
	Tester	0.099	-38.51%	0.527	0.109	0.891	0.055	1.7	2.4	4.1
	HoarePrompt	0.250	+55.28%	0.601	0.895	0.105	0.692	144.3 (106.8)	27.3	171.6 (134.1)
	HoarePrompt no unroll	0.209	+29.81%	0.588	0.858	0.142	0.681	12.0 (9.5)	1.7	13.7 (11.2)

Abbreviations: MCC = Matthews Correlation Coefficient, Δ MCC: Relative MCC Improvement over Vanilla, BA = Balanced Accuracy, TPR = True Positive Rate, FNR = False Negative Rate, FPR = False Positive Rate, IT = Input Tokens, OT = Output Tokens, TT = Total Tokens. Parentheses in IT and TT for HoarePrompt classifiers indicate token counts excluding few-shot learning examples, which can be eliminated via fine-tuning.

blocks—to reduce prompt size while maintaining clarity. In pilots, full-state annotation improved MCC over Zero-shot-CoT by 19.7%, whereas our sparse strategy achieved 62%. We hypothesize that excessive detail dilutes focus, while key transitions aid reasoning.

5.3 RQ1: Classification Performance

In the first research question, we examine whether structural reasoning with descriptions of reachable states helps HoarePrompt improve LLMs’ ability to evaluate program correctness compared to other techniques. By systematically tracking program states through HoarePrompt, we hypothesize that structured reasoning will enhance correctness classification.

Table 3 shows the results for each model. In comparison with LLM-based baselines, Vanilla and Zero-shot-CoT, HoarePrompt consistently demonstrated higher classification quality. On average, HoarePrompt improves the MCC by 61% over Zero-shot-CoT and 72% over Vanilla. The impact of HoarePrompt is most pronounced in smaller models. The most significant improvement of MCC is in Qwen2.5-7B, being 87% over Zero-shot-CoT and 189% over Vanilla. For Qwen2.5-72B, MCC improves by 34% and 53% over the Zero-shot-CoT and Vanilla respectively. The results of the Llama model follow similar trends to those exhibited by the Qwen2.5 models, being especially close to Qwen2.5-72b, which is of a similar size.

Similar improvement is observed across other classification metrics. HoarePrompt’s TPR is 15.5% higher than Zero-shot-CoT and 107.1% higher than Vanilla. HoarePrompt also reduces the FNR

compared to Zero-shot-CoT and Vanilla, The FPR for HoarePrompt (0.502), is also just slightly higher than Zero-shot-CoT (0.459) but significantly higher than Vanilla (0.254).

HoarePrompt also significantly outperforms the Tester classifier, increasing MCC by 106% on average. Notably, Tester achieves a very low average TPR of only 6.7%, indicating frequent misclassifications of correct programs. This issue stems from test generation inaccuracies: LLMs often fail to produce correct output oracles, leading to false negatives. Table 4 shows the breakdown of Tester behavior. On average, each program was evaluated with 13–28 test cases, but 15–25% of examples had no passing test cases, and only 3–8% had no failing test. Thus, the program was marked incorrect in over 90% of cases, including the vast majority of correct programs, due to at least one failing test case. This affected all model sizes, showcasing a limitation of test-based correctness classification. A qualitative analysis of the misclassifications of HoarePrompt and Tester are presented in Section 5.7.

RQ1: HoarePrompt consistently outperforms the baselines in terms of classification quality across all LLMs, achieving an average improvement of **61%** in MCC over Zero-shot-CoT, **72%** over Vanilla and **106%** over Tester.

5.4 RQ2: Impact of Inductive Reasoning

Loops introduce complexity for reasoning about programs. HoarePrompt mitigates this complexity by inductively reasoning about

Table 4: Tester statistics across four LLMs. “Avg. Total” and “Avg. Passed” — the average number of generated/passed tests per program. “0 Pass %” and “0 Fail %” — the percentage of programs for which all tests failed or all tests passed.

Model	Avg. Total	Avg. Passed	0 Pass %	0 Fail %
Qwen2.5-7B	27.81	9.99	23.17%	3.32%
Qwen2.5-32B	17.30	7.86	15.14%	3.63%
Qwen2.5-72B	17.47	6.50	16.90%	3.13%
LLaMA3.1-70B	12.72	5.02	25.34%	8.19%

loops using our novel few-shot-driven k -induction technique. This technique involves unrolling loops k times, with $k = 3$ in our experiments, before generalizing the final state.

This question investigates whether loop unrolling enhances correctness classification quality. To measure the impact of our few-shot-driven k -induction algorithm, we compare HoarePrompt’s classification performance with that of HoarePrompt (no unroll) that summarizes loops within a single step.

As shown in Table 3, the few-shot-driven k -induction technique provides significant additional performance gains, improving the MCC by an average of 25.7% over the non-inductive version (HoarePrompt no unroll). Specifically, it improves MCC by 33.6% in Qwen2.5-7B, 45.3% in Qwen2.5-Coder-32B, 12.1% in Qwen2.5-32B and 19.6% for LLaMA3.1-70b. Even without unrolling, HoarePrompt demonstrates the second-best result among the studied classifiers.

RQ2: Few-shot-driven k -induction enhances classification performance across all models, improving the MCC by an average of 25.7% over the non-inductive version of HoarePrompt.

5.5 RQ3: Token Usage

Since HoarePrompt involves multiple LLM queries, it consumes more tokens than direct LLM prompts [35]. We quantified this increase and investigated the trade-off between token use and classification quality. Table 3 provides the total token usage across classifiers and models for the 645 problem-solution pairs of the CoCoCLaNeL. For each classifier, we measured the input tokens (IT), the output tokens (OT), and total tokens (TT). As explained in Section 4.2, we chose to implement our natural strongest post-condition computation using prompt engineering with few-shot learning examples due to budget constraints. Tokens representing these hard-coded examples are repeatedly sent to an LLM during the algorithm iterations; in principle, they can be eliminated via fine-tuning. Because of that, for the HoarePrompt variants, we also report the number of “essential” input and total token, in parentheses, that excludes hard-coded few-shot example tokens.

On average, HoarePrompt consumes 85.9 times more tokens than Vanilla (51 times when excluding non-essential tokens), 64.1 times more tokens than Zero-shot-CoT (38.6 times more when excluding non-essential tokens), and 40.8 times more tokens than Tester (24.6 times more, when excluding non-essential tokens).

A key reason for the increased token usage is our inductive reasoning algorithm (few-shot-driven k -induction) that performs loop unrolling. On average, it requires 10.9 times extra tokens (8.6

Table 5: Code generation results on 100 problems from CoCoCLaNeL. Correct%: percentage of programs passing all hidden tests; ATP: average fraction of test cases passed.

Method	Llama3.1		GPT-4.1	
	Correct%	ATP	Correct%	ATP
HoarePrompt	33%	57.98%	47%	69.55%
Testing	29%	54.89%	42%	69.53%
Baseline	30%	56.34%	37%	62.03%

times extra when excluding non-essential tokens) than the non-inductive variant. Thus, the few-shot k -induction is more suitable for situations when classification accuracy is prioritized over cost. The non-inductive version, HoarePrompt no unroll, represents a practical trade-off by retaining a substantial performance boost (27.8% better than the top performing approach, Zero-shot-COT) while consuming 90% fewer tokens than full unrolling. Overall, it consumes only 6.4 times more tokens (4.5 times more, when excluding non-essential tokens) than Zero-shot-COT.

RQ3: HoarePrompt’s significant performance boost comes at the expense of an average token usage increase of 64 times (39 times when excluding non-essential tokens that can be eliminated via fine-tuning) compared to the most accurate baseline, Zero-shot-COT. The version of HoarePrompt without unrolling represents a practical trade-off between classification quality and cost, demonstrating the second-best performance, while consuming 90% fewer tokens than the main variant.

5.6 RQ4: Code generation

To assess HoarePrompt as a feedback mechanism in the context of code generation, we adapted the AgentCoder [24] pipeline and implemented two variants: one using test-based feedback (as in the original), and one replacing test failures with correctness judgments from HoarePrompt. The third baseline generates a final solution in a single step without refinement. We evaluated all three methods on 100 problems randomly sampled from CoCoCLaNeL. We excluded problems with multiple valid outputs — cases where different programs produce acceptable but non-identical outputs — since correctness becomes ambiguous and not reliably testable [42].

For our main experiments, we used LLaMA3.1-70b-instruct. Due to the complexity of CoCoCLaNeL and to evaluate generalization on more capable models, we additionally employed the stronger GPT-4.1 model. Results for both models are shown in Table 5.

With LLaMA3.1-70B, the testing-based variant occasionally regressed due to incorrect oracles, reducing both fully correct programs and test pass rates compared to the baseline. In contrast, HoarePrompt consistently improved both metrics, highlighting its robustness. The Appendix includes a regression example and HoarePrompt’s improvement. With the stronger GPT-4.1, the test-based variant improves over the baseline — suggesting that test-driven refinement benefits from stronger models. However, HoarePrompt still outperforms both, achieving the highest correct rate (47%) and best ATP (69.55%).

RQ4: HoarePrompt is effective as a feedback mechanism for iterative code generation, increasing the number of solved problems by 12–14% compared to using generated tests as feedback.

5.7 Qualitative Error Analysis

We manually audited misclassifications to understand failure modes of HoarePrompt and Tester. First for HoarePrompt:

- **Missing Edge Case:** The NSP chain omits critical boundary conditions, leading to overly optimistic judgments.
- **Oversimplified Annotations:** Annotations blur fine-grained state differences, producing misleading correctness cues.
- **Incorrect Loop Generalization:** Loop effects are overgeneralized, resulting in faulty postconditions.
- **Semantic Trust Bias:** When code structure resembles the description, the LLM sometimes ignores contradictory annotations and falsely assumes correctness.
- **Interprocedural Failures:** Reasoning errors occur across function boundaries, corrupting inference consistency.
- **Variable Scope or Shadowing Errors:** In loop-heavy code, reused indices or scope bleed distort postconditions.
- **Description–Code Misalignment (rare):** For correct programs using alternative logic patterns (e.g., DP vs. greedy), annotations fail to reflect intent.

and for Tester, false negatives fell in four categories:

- **Misread semantics (19%):** misunderstood IO relation.
- **Output computation failure (58%):** logic is described correctly but numerical results are wrong.
- **Multiple valid answers (21%):** tests assert one of several correct outputs
- **Insufficient coverage (2%):** corner cases not tested.

In Figure 7, the program (a greedy method with a suffix array) is correct. VANILLA misreads the role of `min(a[i], b[i])`; Tester predicts an incorrect output—consistent with findings that LLMs struggle to directly compute nontrivial results [18]. HoarePrompt’s annotations make the purpose of `c` explicit (per-position minimum cost), enabling the correct verdict.

6 Threats of Validity

LLMs are inherently nondeterministic [41], leading to varied responses. To address this, we employed multiple reruns per sample with confidence-based sampling to ensure robust performance estimates. Although further reruns could slightly refine our results, the current methodology provides high-confidence evaluations.

Main threats to external validity are that the selected dataset may not generalize to other programs, or that performance enhancements will not generalize to other LLMs. To avoid data leakage, we constructed a new dataset from recent Codeforces competition problems, ensuring challenging, human-authored, and varied submissions. Despite efforts to minimize biases, future studies on alternative datasets could further validate generalizability and performance trends. Apart from that, real-world requirements contain ambiguities, which are typically not present in programming competition problems. Techniques like ClarifyGPT [36] that detects ambiguities in informal requirements may address this concern.

We evaluated HoarePrompt across models of diverse sizes (7B to 72B) from 2 families (Qwen, Llama), including Qwen-32B-Coder, a model specialized for coding tasks. This broad spectrum enhances generalizability; however, different LLM architectures may yield varying results. Nevertheless, observed trends suggest broad applicability, especially relevant given industry shifts toward proprietary small and mid-sized models [7]. Evaluating HoarePrompt on larger and more powerful models may help assess its scalability and effectiveness in leveraging more advanced reasoning capabilities.

7 Related Work

HoarePrompt introduces a new paradigm for reasoning about programs in NL. Its contributions are relevant to works on reasoning about code with LLMs, LLM-assisted testing and verification.

Reasoning About Code with LLMs. Many generic approaches have been introduced to realize effective chain-of-thought (CoT) reasoning [51]. We employed Zero-shot-CoT [30], a popular automation of CoT, as a baseline in our experiments. CJ-Eval [59] uses LLMs as judges of program correctness applying an approach equivalent to our baseline Vanilla prompt (Section 4.3). Since CJ-Eval is based on a widely-used Apps dataset [21], which is subject to data leakage, we constructed a new dataset CoCoLaNeL. A large body of work focused on reasoning about concrete execution states. Zhang et al. [58] fine-tuned LLMs with concrete execution states to improve their ability to track execution faithfully. Similarly, NExT [38] enhances debugging using programs annotated with concrete execution traces. CodeMind [34] includes a “specification reasoning” task that measures how including test data in the specification helps the model generate correct code. REval [6] evaluates runtime behaviour reasoning capabilities, specifically, consistency in intermediate state prediction by analyzing variable states at specific execution points. The key difference of HoarePrompt is that it reasons about not individual executions but, potentially, infinitely many executions, without relying of specific inputs.

LLM-Assisted Formal Verification. Recent works on LLM-assisted verification focused on translating natural language specifications into formal ones, and rely on external theorem provers or SMT solvers [16]. Marmaragan [11] synthesizes formal Ada/SPARK contracts from natural language. Similarly, SynVer [37] biases LLM-generated C code towards verifiability by producing assertions and invariants, subsequently verified by Coq or Verified Software Toolchain (VST). LLMLIFT [3] integrates LLMs into verified compilation, generating proof sketches alongside code translation, with formal verification performed via SMT solvers. Stanford et al. [29] introduce Proof-Carrying Code Completions (PC3), prompting LLMs to produce Dafny [31] programs accompanied by machine-checkable correctness proofs, automating verification through Dafny. While these approaches embed formal verification into LLM-driven code generation pipelines, they fundamentally rely on external verification systems such as Dafny, Coq [50], or Z3 [12]. Despite the promise of these methods, they have shown limited scalability, with Marmaragan achieving only 50.7% correctness in its generated Ada/SPARK contracts [11] and Endres et al. [16] reporting that even with optimized prompting, only 77% of LLM-generated postconditions were verifiable. HoarePrompt employs an alternative approach of constructing structured informal proofs in natural language, leveraging its expressive power

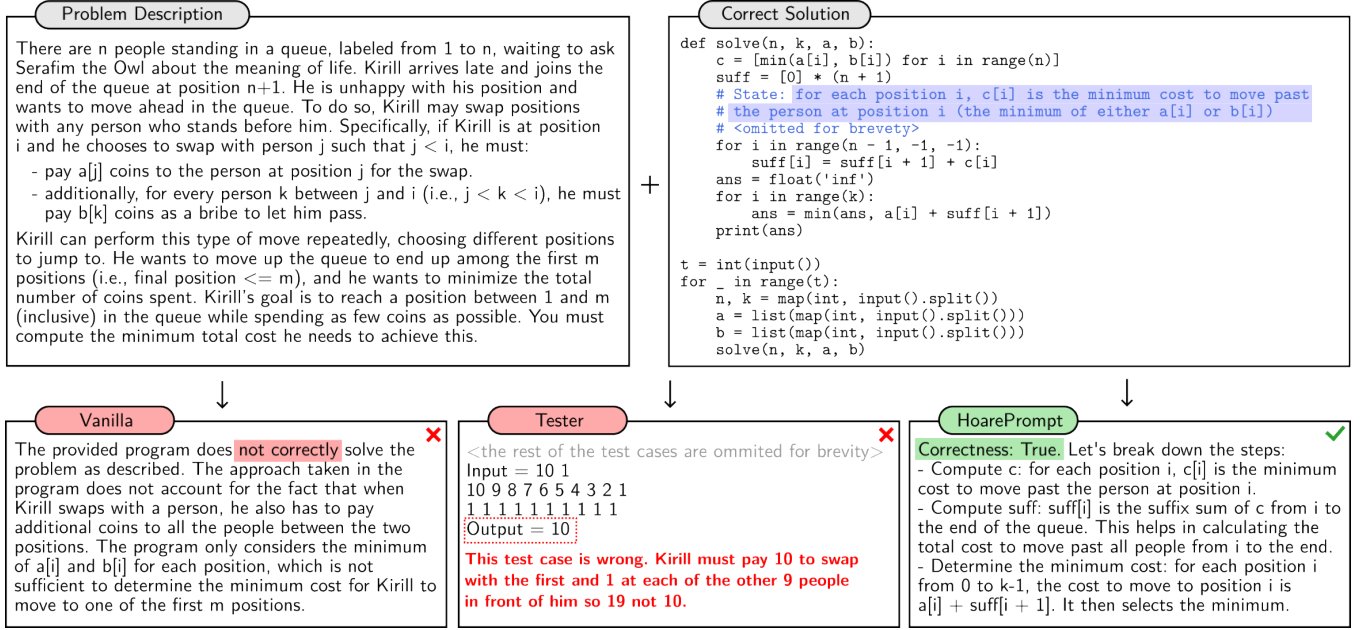


Figure 7: A CoCoClaNeL problem with a correct solution. VANILLA misclassifies by misreading the algorithm; TESTER fails by predicting wrong outputs; HoarePrompt’s state annotations enable a correct judgment.

and flexibility, for classifying program correctness. Following previous work [27] such informal proofs might be translated into formal ones to automate formal verification. Closest to our loop reasoning technique, Liu et al. [33] propose LLM-SE, which combines symbolic execution with LLMs to generate loop invariants for memory-manipulating programs. While HoarePrompt does not target formal invariant synthesis, instead focusing in textual inductive reasoning. Although HoarePrompt does not provide formal guarantees, prior work shows that formal proofs mirror informal reasoning [27], suggesting HoarePrompt could serve as a foundation for automated verification. Integrating HoarePrompt with techniques like symbolic CoT [55] may further enhance reasoning and provide guarantees. A promising future direction is to develop a hybrid (formal and natural) language to promote more grounding of the LLM’s output.

LLM-Based Automated Test Generation Recent works [8, 32, 43, 57] have employed LLMs for automated test generation. Apart from that, the analysis of LLM-generated test outcomes has been used to facilitate program synthesis tasks [54], exemplified by CODET [5], AgentCoder [24], CodeCoT [23], and CoCoST [20]. Section 5 shows that using LLM-generated tests for correctness classification, embodied in our *Tester* baseline adopting AgentCoder’s approach [24], suffers from high false negative rate, as LLMs fail to infer precise assertions for complex requirements. Ideas of HoarePrompt could be synergized with testing to improve accuracy and cost.

Competition-Level Benchmarks Recent studies reveal significant data leakage in widely used LLM benchmarks, inflating model performance due to test-training overlaps [13, 39]. New benchmarks, such as LiveCodeBench [26], CodeElo [42], ProBench [56], and USACO-based evaluations [45], mitigate leakage by incorporating

recent programming competition problems. However, they primarily assess code generation capabilities and do not provide solution submissions with correctness labels. In contrast, CoCoClaNeL specifically targets correctness classification by offering labeled problem-solution pairs sourced from Codeforces 2024 contests, enabling evaluation of LLMs on their execution-state reasoning capabilities rather than memorization or test-based validation alone.

8 Conclusion

This work introduces HoarePrompt, the first LLM-based program analysis approach to assess program correctness against natural language requirements. Unlike traditional verification and testing that depend on external tools or concrete test executions, HoarePrompt systematically applies natural language-based structural reasoning inspired by the strongest postcondition calculus and k -induction. Our evaluation shows significant performance improvements in classification quality, with HoarePrompt increasing the MCC by 61% over Zero-shot-CoT, 72% over Vanilla, and 106% over an LLM testing-based classifier. Our novel k -induction further enhances performance, yielding an average MCC improvement of 26% compared to a non-inductive approach. Although HoarePrompt consumes more tokens, the no-unroll variant provides a practical trade-off, achieving a 28% MCC improvement over Zero-shot-CoT while using 90% fewer tokens than the standard HoarePrompt version.

Acknowledgments

This work was sponsored by the National Key Research and Development Program of China under Grant No. 2022YFB4501902 and

the National Natural Science Foundation of China (NSFC) under Grant No. W2411051.

References

- [1] Simran Arora, Avaniika Narayan, Mayee F Chen, Laurel Orr, Neel Guha, Kush Bhatia, Ines Chami, Frederic Sala, and Christopher Ré. 2022. Ask me anything: A simple strategy for prompting language models. *arXiv preprint arXiv:2210.02441* (2022).
- [2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).
- [3] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. 2024. Verified Code Transpilation with LLMs. *arXiv:2406.03003 [cs.PL]* <https://arxiv.org/abs/2406.03003>
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. Code: Code generation with generated tests. *arXiv preprint arXiv:2207.10397* (2022).
- [6] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2024. Reasoning Runtime Behavior of a Program with LLM: How Far Are We? *arXiv:2403.16437 [cs.SE]* <https://arxiv.org/abs/2403.16437>
- [7] Lihu Chen and Gaël Varoquaux. 2024. What is the role of small models in the llm era: A survey. *arXiv preprint arXiv:2409.06857* (2024).
- [8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.
- [9] Davide Chicco and Giuseppe Jurman. 2023. The Matthews correlation coefficient (MCC) should replace the ROC AUC as the standard metric for assessing binary classification. *BioData Mining* 16, 1 (2023), 4. <https://doi.org/10.1186/s13040-023-00322-4>
- [10] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 238–252.
- [11] Marcos Cramer and Lucian McIntyre. 2025. Verifying LLM-Generated Code in the Context of Software Verification with Ada/SPARK. *arXiv:2502.07728 [cs.SE]* <https://arxiv.org/abs/2502.07728>
- [12] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [13] Chunyuan Deng, Yilun Zhao, Yuzhao Heng, Yitong Li, Jiannan Cao, Xian-gru Tang, and Arman Cohan. 2024. Unveiling the Spectrum of Data Contamination in Language Model: A Survey from Detection to Remediation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 16078–16092. <https://doi.org/10.18653/v1/2024.findings-acl951>
- [14] Edsger Wybe Dijkstra, Edsger Wybe Dijkstra, Edsger Wybe Dijkstra, Etats-Unis Informaticien, and Edsger Wybe Dijkstra. 1976. *A discipline of programming*. Vol. 613924118. prentice-hall Englewood Cliffs.
- [15] Alastair F Donaldson, Leopold Haller, Daniel Kroening, and Philipp Rümmer. 2011. Software verification using k-induction. In *Static Analysis: 18th International Symposium, SAS 2011*. Springer, 351–368.
- [16] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1889–1912.
- [17] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3—where programs meet provers. In *Programming Languages and Systems: 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16–24, 2013. Proceedings 22*. Springer, 125–128.
- [18] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International Conference on Machine Learning*. PMLR, 10764–10799.
- [19] Alibaba Group. 2024. Qwen2.5: Advancing Open-Source Large Language Models. https://qwenlm.github.io/blog/qwen2.5-llm/?utm_source=chatgpt.com
- [20] Xinyi He, Jiaru Zou, Yun Lin, Mengyu Zhou, Shi Han, Zejian Yuan, and Dongmei Zhang. 2024. CoCoST: Automatic Complex Code Generation with Online Searching and Correctness Testing. *arXiv preprint arXiv:2403.13583* (2024).
- [21] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence with APPS. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.
- [22] Charles Antony Richard Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (1969), 576–580.
- [23] Dong Huang, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Codcot: Tackling code syntax errors in cot reasoning for code generation. *arXiv preprint arXiv:2308.08784* (2023).
- [24] Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010* (2023).
- [25] Yiming Huang, Zhenghao Lin, Xiao Liu, Yeyun Gong, Shuai Lu, Fangyu Lei, Yaobo Liang, Yelong Shen, Chen Lin, Nan Duan, and Weizhu Chen. 2024. Competition-Level Problems are Effective LLM Evaluators. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Bangkok, Thailand, 13526–13544. <https://doi.org/10.18653/v1/2024.findings-acl803>
- [26] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv:2403.07974 [cs.SE]* <https://arxiv.org/abs/2403.07974>
- [27] Albert Q Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2022. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283* (2022).
- [28] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Soneya Hossain, Baishakhi Ray, Varun Kumar, Xiaofei Ma, and Anoop Deoras. 2024. LeDex: Training LLMs to Better Self-Debug and Explain Code. *Advances in Neural Information Processing Systems* 37 (2024), 35517–35543.
- [29] Parnian Kamran, Premkumar Devanbu, and Caleb Stanford. 2024. Vision Paper: Proof-Carrying Code Completions. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops* (Sacramento, CA, USA) (ASEW '24). Association for Computing Machinery, New York, NY, USA, 35–42. <https://doi.org/10.1145/3691621.3694932>
- [30] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [31] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 348–370.
- [32] Kefan Li and Yuan Yuan. 2024. Large language models as test case generators: Performance evaluation and enhancement. *arXiv preprint arXiv:2404.13340* (2024).
- [33] Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. 2024. Towards General Loop Invariant Generation: A Benchmark of Programs with Memory Manipulation. *arXiv:2311.10483 [cs.PL]* <https://arxiv.org/abs/2311.10483>
- [34] Changshu Liu, Shizhuo Dylan Zhang, Ali Reza Ibrahimzade, and Reyhaneh Jabbarvand. 2024. Codemind: A framework to challenge large language models for code reasoning. *arXiv preprint arXiv:2402.09664* (2024).
- [35] Tongxuan Liu, Xingyu Wang, Weizhe Huang, Wenjiang Xu, Yuting Zeng, Lei Jiang, Hailong Yang, and Jing Li. 2024. Groupdebate: Enhancing the efficiency of multi-agent debate using group discussion. *arXiv preprint arXiv:2409.14051* (2024).
- [36] Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binquan Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. Clarifygpt: Empowering llm-based code generation with intention clarification. *arXiv preprint arXiv:2310.10996* (2023).
- [37] Prasita Mukherjee and Benjamin Delaware. 2024. Towards Automated Verification of LLM-Synthesized C Programs. *arXiv:2410.14835 [cs.PL]* <https://arxiv.org/abs/2410.14835>
- [38] Ansong Ni, Miltiadis Allamanis, Arman Cohan, Yinlin Deng, Kensen Shi, Charles Sutton, and Pengcheng Yin. 2024. Next: Teaching large language models to reason about code execution. *arXiv preprint arXiv:2404.14662* (2024).
- [39] Shiwen Ni, Xiangtao Kong, Chengming Li, Xiping Hu, Ruifeng Xu, Jia Zhu, and Min Yang. 2025. Training on the Benchmark Is Not All You Need. *arXiv:2409.01790 [cs.CL]* <https://arxiv.org/abs/2409.01790>
- [40] Flemming Nielson, Hanne R Nielson, and Chris Hankin. 2015. *Principles of program analysis*. springer.
- [41] Shuyin Ouyang, Jie M Zhang, Mark Harman, and Meng Wang. 2025. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–28.
- [42] Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. 2025. CodeElo: Benchmarking Competition-level Code Generation of LLMs with Human-comparable Elo Ratings. *arXiv:2501.01257 [cs.CL]* <https://arxiv.org/abs/2501.01257>
- [43] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.

- [44] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2023. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324* (2023).
- [45] Quan Shi, Michael Tang, Karthik Narasimhan, and Shunyu Yao. 2024. Can Language Models Solve Olympiad Programming? *arXiv:2404.10952* [cs.CL]
- [46] Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. 2018. Learning loop invariants for program verification. *Advances in Neural Information Processing Systems* 31 (2018).
- [47] LLM Stats. 2024. *GPT-4o-mini 2024-07-18 vs Qwen2 72B Instruct*. <https://llm-stats.com/models/compare/gpt-4o-mini-2024-07-18-vs-qwen2-72b-instruct> Accessed: 2025-03-14.
- [48] Rob Sullivan and Nelly Elsayed. 2024. Can Large Language Models Act as Symbolic Reasoners? *arXiv:2410.21490* [cs.CL] <https://arxiv.org/abs/2410.21490>
- [49] Shin Hwei Tan, Jooyong Yi, Sergey Mechtaev, Abhik Roychoudhury, et al. 2017. Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*. IEEE, 180–182.
- [50] The Coq Development Team. 2024. The Coq Reference Manual – Release 8.19.0. <https://coq.inria.fr/doc/V8.19.0/refman>.
- [51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [52] Chulin Xie, Yangsibo Huang, Chiyuan Zhang, Da Yu, Xinyun Chen, Bill Yuchen Lin, Bo Li, Badih Ghazi, and Ravi Kumar. 2024. On memorization of large language models in logical reasoning. *arXiv preprint arXiv:2410.23123* (2024).
- [53] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. *arXiv:2306.13063* [cs.CL] <https://arxiv.org/abs/2306.13063>
- [54] Weimin Xiong, Yiwen Guo, and Hao Chen. 2023. The program testing ability of large language models for code. *arXiv preprint arXiv:2310.05727* (2023).
- [55] Jundong Xu, Hao Fei, Liangming Pan, Qian Liu, Mong-Li Lee, and Wynne Hsu. 2024. Faithful logical reasoning via symbolic chain-of-thought. *arXiv preprint arXiv:2405.18357* (2024).
- [56] Lei Yang, Renren Jin, Ling Shi, Jianxiang Peng, Yue Chen, and Deyi Xiong. 2025. ProBench: Benchmarking Large Language Models in Competitive Programming. *arXiv:2502.20868* [cs.CL] <https://arxiv.org/abs/2502.20868>
- [57] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? evaluating and improving chatgpt for unit test generation. *arXiv preprint arXiv:2305.04207* (2023).
- [58] David W. Zhang, Michaël Defferrard, Corrado Rainone, and Roland Memisevic. 2025. Grounding code understanding in step-by-step execution. <https://openreview.net/forum?id=MUr7FI93QS>
- [59] Yuwei Zhao, Ziyang Luo, Yuchen Tian, Hongzhan Lin, Weixiang Yan, Annan Li, and Jing Ma. 2024. CodeJudge-Eval: Can Large Language Models be Good Judges in Code Understanding? *arXiv preprint arXiv:2408.10718* (2024).